University of Cape Town



Machine Learning Assignment 2

Pamela Afful

Student Number: AFFPAM001

October 30, 2026

Department of Statistical Sciences Plagiarism Declaration form

A copy of this form, completed and signed, to be attached to all coursework submissions to the Statistical Sciences Department. Submissions without this form will not be marked.

COURSE CODE: **STA** 5068Z

COURSE NAME: MACHINE LEARNING

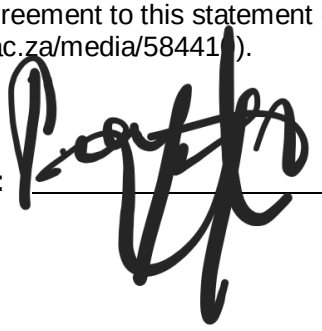
STUDENT NAME: PAMELA AFFUL

STUDENT NUMBER: AFFPAM001

PLAGIARISM DECLARATION

1. I know that plagiarism is wrong. Plagiarism is to use another's work and pretend that it is one's own.
2. I have used a generally accepted citation and referencing style. Each contribution to, and quotation in, this tutorial/report/project from the work(s) of other people has been attributed, and has been cited and referenced.
3. This tutorial/report/project is my own work.
4. I have not allowed, and will not allow, anyone to copy my work with the intention of passing it off as his or her own work.
5. I acknowledge that copying someone else's assignment or essay, or part of it, is wrong, and declare that this is my own work.

Note that agreement to this statement does not exonerate you from UCT's Academic Misconduct policy (https://uct.ac.za/media/58441).

Signature: 

Date: 18/10/2026

Drone Escape

a)

Game Status

```
set.seed(2024)

game_status = function(xt, objects, rad, center =
 ↪  matrix(c(0,0), nrow=1, ncol=2), outer_rad = 1)
{
  J         = nrow(objects)
  ones      = matrix(1, J, 1)
  dists     = sqrt(rowSums((objects - ones %*% xt)^2))
 ↪  # distance btwn drone and ALL obj
  min_dist = min(dists) # dist to closest tree

  outer_circle_point_dist = sqrt(sum((center -
 ↪  xt)^2)) # distance between circle center and
 ↪  drone

  if (min_dist < rad) {
    status = -1                          # hit a tree/obj
  } else if (outer_circle_point_dist > outer_rad) {
    status = 1                           # escaped
  } else {
    status = 0                           # keep going
  }

  list(status = status, closest =
   ↪  objects[which.min(dists), , drop = FALSE],
   ↪  min_dist = min_dist)
}

#game_status(x0, objects, rad, center =
 ↪  matrix(c(0,0), nrow=1, ncol=2), outer_rad = 1)
```

b) Experimental Setup 1

i)

```
#| layout-ncol: 2
#| fig-subcap:
#|   - "GA Parameter Search Trace Plot"
#|   - "Trajectories over 100 initial positions"

set.seed(2024)

draw_starts = function(N = 1)
{
    r_x      =  runif(N,0,0.25)
    pi_x     =  runif(N,-1,1)*pi
    xt       =
 ↪  matrix(cbind(r_x*cos(pi_x),r_x*sin(pi_x)),nrow =
 ↪  N,byrow = TRUE)
    return(xt)
}

p     = 2
q     = 2
nodes =c(3,3)
npars =
 ↪  p*nodes[1]+nodes[1]*nodes[2]*+nodes[2]*q+nodes[1]+nodes[2]+q
 ↪  # total num param
theta_rand = runif(npars,-1,1)

#####################NN
 ↪  MOdel##############################

model = function(xt, theta, nodes = c(3,3))
{
```

```
  # Infer dimensions:
  N = dim(xt)[1]
  p = dim(xt)[2]    # number of input features/coords
  q = 2             # output steps
  dims = c(p, nodes, q)    # p -> 3 -> 3 -> 2

  # Populate weight matrices:
  index = 1:(dims[1]*dims[2])
  W1    = matrix(theta[index], dims[1], dims[2])
  index = max(index) + 1:(dims[2]*dims[3])
  W2    = matrix(theta[index], dims[2], dims[3])
  index = max(index) + 1:(dims[3]*dims[4])
  W3    = matrix(theta[index], dims[3], dims[4])

  # Populate bias vectors:
  index = max(index) + 1:dims[2]
  b1    = matrix(theta[index], dims[2], 1)
  index = max(index) + 1:dims[3]
  b2    = matrix(theta[index], dims[3], 1)
  index = max(index) + 1:dims[4]
  b3    = matrix(theta[index], dims[4], 1)

  ones = matrix(1, N, 1)
  a0   = t(xt)

  # Evaluate the updating equation in matrix form
  a1 = tanh(t(W1) %*% a0 + b1 %*% t(ones))    # hidden
 ↪  layer 1
  a2_hidd = tanh(t(W2) %*% a1 + b2 %*% t(ones))    #
 ↪  hidden layer 2
  a3 = tanh(t(W3) %*% a2_hidd + b3 %*% t(ones))    #
 ↪  output layer

  # Return a list of relevant objects:
  return(list(a2 = matrix(a3, ncol = 2, nrow = 1)))
}
```

```
#################### plot game function
plot_game =
 ↪  function(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2),inner
{
  plot(NULL, type ='n', col =
   ↪  'darkgreen',xlim=c(-2,2),ylim=c(-2,2))

  # plot inner and outer circles

  theta = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(theta)
  y_inner = center[2] + inner_rad * sin(theta)


  x_outer = center[1] + outer_rad * cos(theta)
  y_outer = center[2] + outer_rad * sin(theta)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

  points(xt[,2]~xt[,1],pch = 9,col = 'black')

   ↪  points(objects[,1],objects[,2],pch=19,col='darkgreen')
  abline(v = c(-1,1), lty = 3,col = 'black',lwd = 2)
  abline(h = c(-1,1), lty = 3,col =
   ↪  c('black','black'), lwd = 2)
}


#################################### game_play
```

```
# play a single game

play_a_game = function(xt,delt,objects,rad,theta,plt
 ↪  = F,trace =
 ↪  TRUE,nodes,inner_rad=0.25,outer_rad=1)
{
  k           =   1  # Count how many iterations
  xt          =   xt # Set the initial coordinate(s)
 ↪  for the drone.
  trajectory = matrix(0,2,100) # store the the
 ↪  postions that the drone takes through out the
 ↪  game/how it moves
  trajectory[,1] = t(x0)

  # Check the game status:
  res_status  = game_status(xt,objects,rad)

  if(plt)
  {

       ↪  plot_game(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2
  }
  while((res_status$status==0)&(k<100)) # single
   ↪  game=100 steps
  {
    k = k+1

    # Now, let the model update the position of the
     ↪  pieces:
    ct = model(xt,theta,nodes)$a2 #res_status$closest
    xt =  xt+ct*delt

    #print(paste0('k: ',k,'xt_new: ',xt))
```

```
    # Checkk the game status after the positions are
     ↪ updates:

    res_status  = game_status(xt,objects,rad)

    trajectory[,k] = t(xt)

  }
  trajectory = trajectory[,1:k,drop = FALSE]
    if(plt)
  {
    lines(trajectory[2,]~trajectory[1,],col =
     ↪ 'darkgrey')
  }

  dmin_final=sqrt(sum((res_status$closest - xt)^2))

  return(list(k = k, status = res_status$status,xt=
   ↪ xt,trajectory = trajectory,dmin=dmin_final))
}

#play_a_game(draw_starts(1),delt,objects,rad,theta_rand,nodes=c(3,3),
 ↪ = F)



############ Obj func
play_T_games = function(theta) # This is our obj
 ↪ function!
{
  inner_rad_in=0.25;outer_rad_in=1;T_games=100;J=50

     # num trees/objects to avoid
  score=0
    for(i in 1:T_games)
    {
```

```
  objects =  objects # keeo constant for now
 #starting position
    xt_try=draw_starts(1)
    #apply across x_try
   res_play
 ↪  =play_a_game(xt_try,delt,objects,rad,theta,plt =
 ↪  FALSE,trace =
 ↪  FALSE,nodes=c(3,3),inner_rad=inner_rad_in,outer_rad=outer_rad_in)
   score  = score+(res_play==+1) # number of times
 ↪  when we got to the other side
    }
  return(score/T_games) # objective to maximise
}

###################GA function to get
 ↪  weights#################
# Use GA function

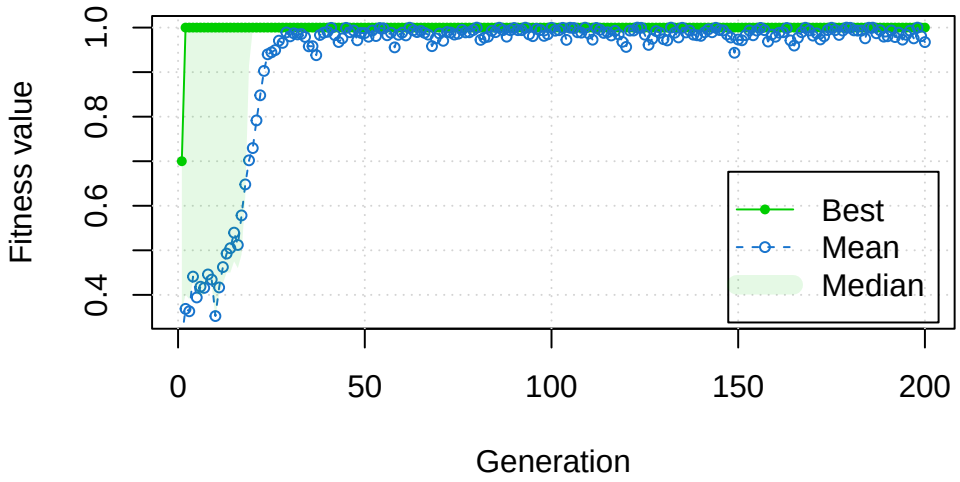
library('GA')

# Read saved RDS From GA code below

# GA <- ga(type   = 'real-valued',
#          fitness = play_T_games, # used here to get
 ↪  the optimal theta
#          lower   = rep(-10,npars), # lower bound
 ↪  vector
#          upper   = rep(+10,npars), # upper bound
 ↪  vect
#          popSize = 100,
#          maxiter = 200) # nr generations
#


#saveRDS(GA,file='ga_results2.rds')
```

```
GA=readRDS('ga_results2.rds')
plot(GA)
```




```
theta_opt=GA@solution[1,]

# Draw race over 100 random starts
tries=draw_starts(100)
center=c(0,0)

plot(NULL, xlim=c(-2,2),ylim=c(-2,2),xlab='position
 ↪  coord 1',ylab='position coord 2',main='Trace Plot
 ↪  for 100 Trajectories: Fixed Tree Configuration')

  thetas = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(thetas)
  y_inner = center[2] + inner_rad * sin(thetas)
```

```
  x_outer = center[1] + outer_rad * cos(thetas)
  y_outer = center[2] + outer_rad * sin(thetas)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

  # points(xt[,2]~xt[,1],pch = 9,col = 'black')

   ↪  points(objects[,1],objects[,2],pch=19,col='darkgreen')

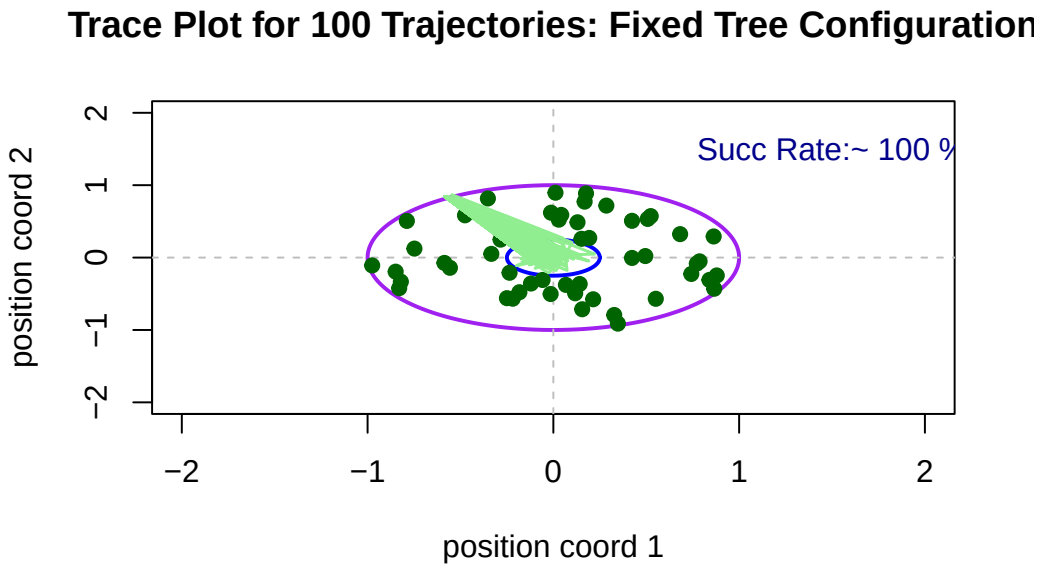

# play the game 100 times with the optimal theta
scores=c() # store results of each game
for(i in 1:dim(tries)[1]){
  xt_i=matrix(tries[i,],nrow=1,ncol=2)

 ↪  res=play_a_game(xt_i,delt,objects,rad,theta=theta_opt,plt
 ↪  = F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
 scores=c(scores,res$status)
  traj= res$trajectory
  colr=ifelse(res$status==1,'lightgreen','brown') #
 ↪  green if succ
  lines(traj[1,]~traj[2,],lwd=1.5,col=colr)

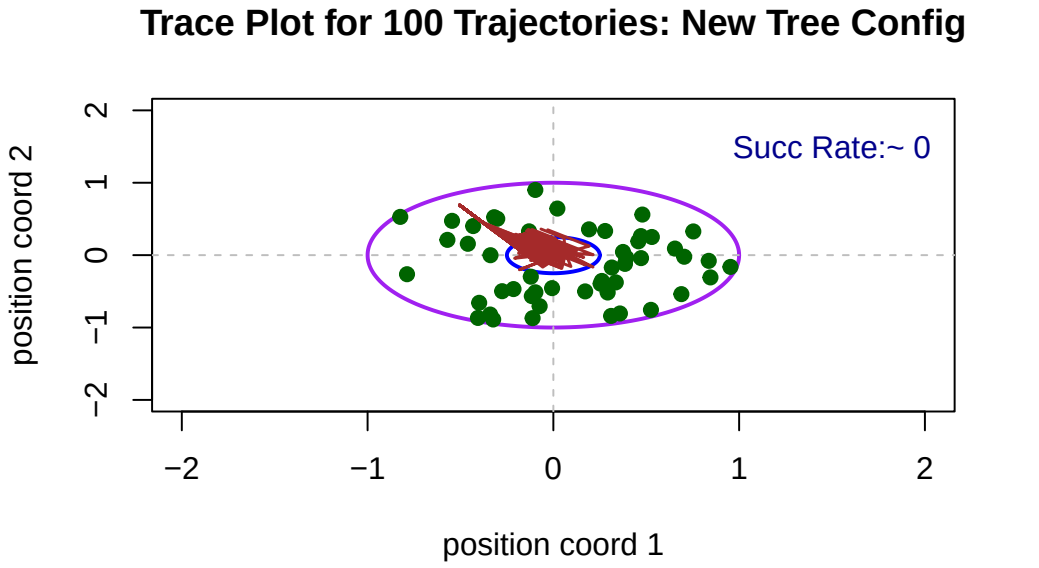
  }

# %Successful exits
sr=(length(which(scores==1))/length(scores))*100 #98%

text(x = 1.5, y = 1.5, labels = paste("Succ
 ↪  Rate:~",round(sr,3),'%'), col = "darkblue", cex =
 ↪  1, font = 1)
```


Trace Plot for 100 Trajectories: Fixed Tree Configuration
Succ Rate:~ 100 %
position coord 2
position coord 1
−2
−1
0
1
2

ii)


Trace Plot for 100 Trajectories: New Tree Config
Succ Rate:~ 0
position coord 2
position coord 1
−2
−1
0
1
2

iii) Key Observations:

- The drone has a near perfect success rate at escaping the forest and the learned weights have learnt how to get around a specific configuration of forest across starting positions
- However, with a new configuration of trees, the drones success rate plummets to 0, indicating that the initial model/weights were highly overfit for a specific distributions of tress across the field. Apart from hitting the objects, most of the drone's trajectories do not make it out of the outer circle in the given number of iteration steps (100). The model has not learnt a general pattern of how to escape the field (across object configurations).

c) Experimental Setup 2

i)

```
model_2 = function(xt_input, theta, nodes =
 ↪  c(3,3),add_arg) #add_arg= 1/(dmin-r)
{
  xt=cbind(xt_input,add_arg)
  # Infer dimensions:
  N = dim(xt)[1]
  p = dim(xt)[2]    # number of input features/coords
  q = 2             # output steps
  dims = c(p, nodes, q)    # p -> 3 -> 3 -> 2

  # Populate weight matrices:
  index = 1:(dims[1]*dims[2])
  W1    = matrix(theta[index], dims[1], dims[2])
  index = max(index) + 1:(dims[2]*dims[3])
  W2    = matrix(theta[index], dims[2], dims[3])
  index = max(index) + 1:(dims[3]*dims[4])
  W3    = matrix(theta[index], dims[3], dims[4])
```

```
  # Populate bias vectors:
  index = max(index) + 1:dims[2]
  b1    = matrix(theta[index], dims[2], 1)
  index = max(index) + 1:dims[3]
  b2    = matrix(theta[index], dims[3], 1)
  index = max(index) + 1:dims[4]
  b3    = matrix(theta[index], dims[4], 1)

  ones = matrix(1, N, 1)
  a0   = t(xt)

  # Evaluate the updating equation in matrix form
  a1 = tanh(t(W1) %*% a0 + b1 %*% t(ones))   # hidden
 ↪  layer 1
  a2_hidd = tanh(t(W2) %*% a1 + b2 %*% t(ones))   #
 ↪  hidden layer 2
  a3 = tanh(t(W3) %*% a2_hidd + b3 %*% t(ones))   #
 ↪  output layer

  # Return a list of relevant objects:
  return(list(a2 = matrix(a3, ncol = 2, nrow = 1)))
}


# Modify play_a_game function
# now includes d_min

play_a_game_mod =
 ↪  function(xt,delt,objects,rad,theta,plt = F,trace
 ↪  = TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
{
  k           =   1  # Count how many iterations
  xt          =   xt # Set the initial coordinate(s)
 ↪  for the drone.
  trajectory = matrix(0,2,100) # store the the
 ↪  postions that the drone takes through out the
 ↪  game/how it moves
```

```
 trajectory[,1] = t(xt)

 # Check the game status:
 res_status  = game_status(xt,objects,rad)
 d_min=sqrt(sum((res_status$closest - xt)^2))

 if(plt)
 {

      ↪  plot_game(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2
 }
 while((res_status$status==0)&(k<100)) # single
  ↪  game=100 steps
 {
   k = k+1

   # Now, let the model update the position of the
    ↪  pieces:
   add_arg=(1/(d_min-rad))
   ct = model_2(xt,theta,nodes,add_arg)$a2
↪  #res_status$closest
   xt =  xt+ct*delt

   #print(paste0('k: ',k,'xt_new: ',xt))

   # Checkk the game status after the positions are
    ↪  updates:

   res_status  = game_status(xt,objects,rad)

   trajectory[,k] = t(xt)

 }
 trajectory = trajectory[,1:k,drop = FALSE]
   if(plt)
 {
```

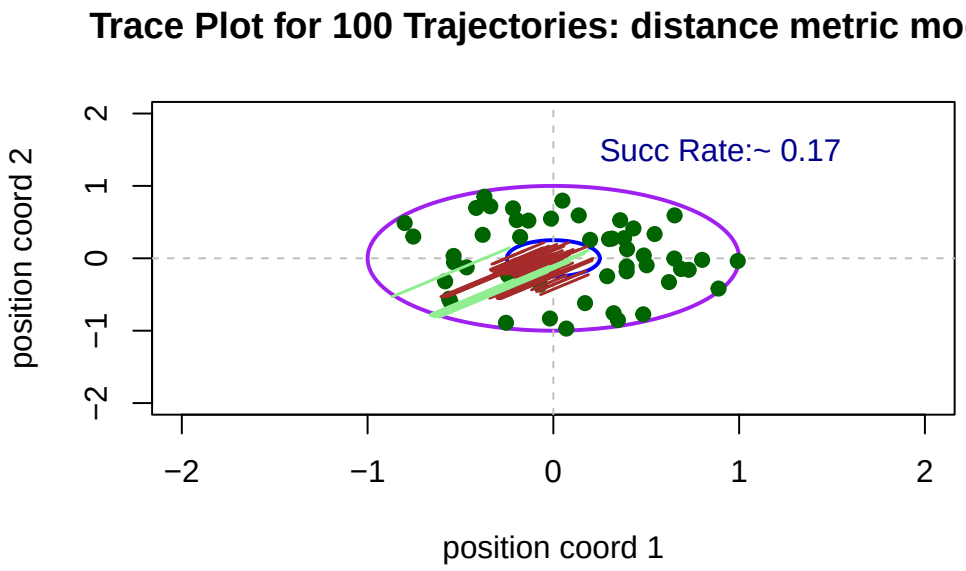
```
    lines(trajectory[2,]~trajectory[1,],col =
     ↪  'darkred',lwd=2)
    #point(trajectory[2,]~trajectory[1,],col =
     ↪  'darkred',lwd=2)
    points(xt[,2]~xt[,1],pch = 9,col = 'black')
  }

  dmin_final=sqrt(sum((res_status$closest - xt)^2))

  return(list(k = k, status = res_status$status,xt=
   ↪  xt,trajectory =
   ↪  trajectory,dmin_final=dmin_final))
}

# Test for a single game
#play_a_game_mod(x0,delt,objects,rad,theta_opt,nodes=c(3,3),plt
 ↪  = T)
```

ii)



The model has an improved success rate of 17% for a new configuration of trees using the distance-augmented model (using the same optimized parameters). The model has adopted a slightly better pattern of avoiding trees, although it still has some noticeable room for improvement. Once again, there are a noticeable number of trajectories that do not make it out of the outer circle in the given number of iterations.

d) Experimental Setup 3

```
 # modified to make a new draw and tree config
play_a_game_mod2 = function(delt,rad,theta,plt =
 ↪  F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
{
  J=50
  # random start
  xt=draw_starts(1) # new random start
  objects =  objects=draw_objects(J)
```

```
 k           =   1  # Count how many iterations
 xt          =   xt # Set the initial coordinate(s)
↪  for the drone.
 trajectory = matrix(0,2,100) # store the the
↪  postions that the drone takes through out the
↪  game/how it moves
 trajectory[,1] = t(xt)

 # Check the game status:
 res_status  = game_status(xt,objects,rad)
 d_min=sqrt(sum((res_status$closest - xt)^2))


 if(plt)
 {

      ↪  plot_game(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2
 }
 while((res_status$status==0)&(k<100)) # single
  ↪  game=100 steps
 {
   k = k+1

   # Now, let the model update the position of the
    ↪  pieces:
   add_arg=(1/(d_min-rad))
   ct = model_2(xt,theta,nodes,add_arg)$a2
↪  #res_status$closest
   xt =  xt+ct*delt

   #print(paste0('k: ',k,'xt_new: ',xt))

   # Checkk the game status after the positions are
    ↪  updates:
```

```
    res_status  = game_status(xt,objects,rad)

    trajectory[,k] = t(xt)

  }
  trajectory = trajectory[,1:k,drop = FALSE]
    if(plt)
  {
    lines(trajectory[2,]~trajectory[1,],col =
     ↪  'darkred',lwd=2)
    #point(trajectory[2,]~trajectory[1,],col =
     ↪  'darkred',lwd=2)
    points(xt[,2]~xt[,1],pch = 9,col = 'black')
  }

  dmin_final=sqrt(sum((res_status$closest - xt)^2))

  return(list(k = k, status = res_status$status,xt=
   ↪  xt,trajectory =
   ↪  trajectory,dmin_final=dmin_final))
}


# Results
set.seed(46)
plot(NULL, xlim=c(-2,2),ylim=c(-2,2),xlab='position
 ↪  coord 1',ylab='position coord 2',main='100
 ↪  Trajectories: Varying Initial Starts and Tree
 ↪  Configs')

  thetas = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(thetas)
  y_inner = center[2] + inner_rad * sin(thetas)
```

```
  x_outer = center[1] + outer_rad * cos(thetas)
  y_outer = center[2] + outer_rad * sin(thetas)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

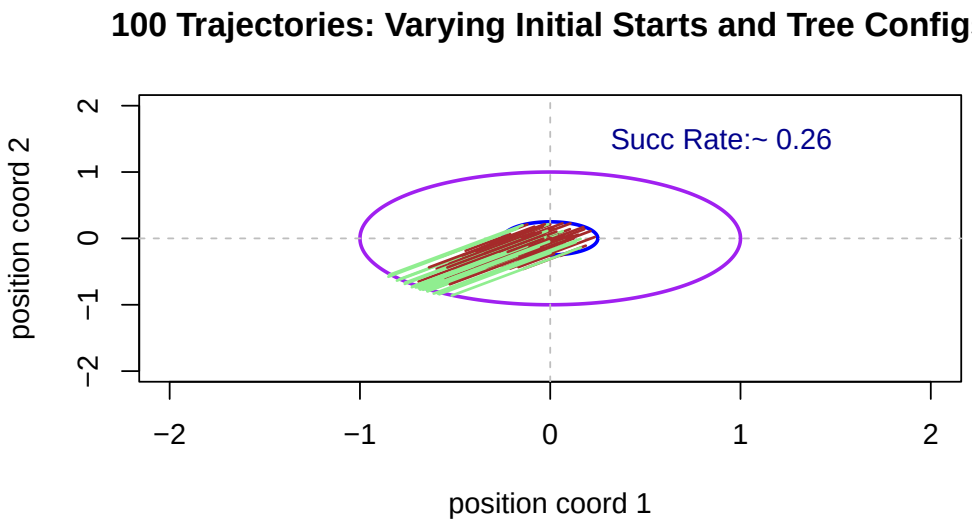
  # points(xt[,2]~xt[,1],pch = 9,col = 'black')

# play the game 100 times with the optimal theta
scores=c() # store results of each game
for(i in 1:dim(tries)[1]){
  res=play_a_game_mod2(delt,rad,theta=theta_opt,plt =
 ↪  F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
 scores=c(scores,res$status)
  traj= res$trajectory
  colr=ifelse(res$status==1,'lightgreen','brown') #
 ↪  green if succ
  lines(traj[1,]~traj[2,],lwd=1.5,col=colr)


  }

# %Successful exits
sr=round(
 ↪  (length(which(scores==1))/length(scores)),3)

text(x = 0.9, y = 1.5, labels = paste("Succ
 ↪  Rate:~",sr), col = "darkblue", cex = 1, font = 1)
```



The success rate has shown a better improvement across variations of the tree configurations and starting points, with a success rate of 26%. There is slight inprovement in generalization for the optimized set of weights.

Monte Carlo Tree Search

a)

```
# m0 = as.matrix(c(0,0,0,0,0,0,0,0,0)) # current game
 ↪  board/flattened
# c(rho(m0,S)$winner ,1)

# Mofified game tree
game_tree_mod= function(m,k,n_moves,strict=F,n_sim=1)
 ↪  #n_moves=target moves
{
    g = c()

    for(sim in 1:n_sim){
```

```
   x=m
   game_state = rho(x,S)

   if(strict){
   condition=(n_moves<=0)}else{ # exactly condition
     condition=((game_state$terminal)|(n_moves<=0))
↪  # up to condition
   }

   if(condition){
        g = c(g,game_state$winner)
        return(g)
   }else{
       index_positions = which(m == 0) # open slot
↪  on the game board/ vector of open

       for(i in 1:length(index_positions))
       {
         x=m
           picked_index=index_positions[i]

           x[picked_index] = k # player's turn

           g  =c(g,
↪  game_tree_mod(x,-1*k,n_moves-1,n_sim=1) )

       }
       }
       return(g)
   }
   }
```

Table 1: Total Wins: 5 vs 8 move game (exact)

	5 exact moves	8 exact moves
#Game Trajectories	16560	405504
Total Wins	1440	127296

Table 2: Player Wins For (up to) 8 move game

	wins	percentage
player 1	49392	0.1218
player 2	77904	0.1921

Player 2 wins 19% of total game trajectories when the game is adjusted for up to 8 moves; in this set up it's better to play second.

b)

```
mcts= function(m,k,alpha,n_sim=1)
{

  g = c()

    for(sim in 1:n_sim){
    x=m
    game_state = rho(x,S)

    if(game_state$terminal)
    {
         g = c(g,game_state$winner)
         return(g)
    }else{
        index_positions = which(m == 0) # open slot
 ↪  on the game board/ vector of open

        for(i in 1:length(index_positions))
```

```
        {
          if (runif(1,0,1) <= alpha) {
          x=m
            picked_index=index_positions[i]

            x[picked_index] = k # player's turn
            g  =c(g, mcts(x,k*-1,alpha) )


          }
        }
        }
        return(g)
    }
    }


# Table Summ

cols=c('Pl 1 Win %','Pl 2 Win %','Ties %')
alphas_=c(0.95,1)

m0=as.matrix(c(+1, +1, 0, 0, 0, 0, 0,0,-1))

pl_1_wins=c()
pl_2_wins=c()
ties=c()
for(alph in alphas_ ){
  if(alph!=1){
  g_sample=mcts(m0,-1,alph)
  }else{
    g_sample=game_tree(m0,-1)
  }

  num_traj=length(g_sample)
  num_wins_player1=round( (length(
 ↪  which(g_sample==-1))/num_traj),4)
```

```
  num_wins_player2=length(
 ↪  which(g_sample==1))/num_traj
  num_ties=length(which(g_sample==0))/num_traj
  pl_1_wins=c(pl_1_wins,num_wins_player1)
  pl_2_wins=c(pl_2_wins,num_wins_player2)
  ties=c(ties,num_ties)
}

mcts_freq=data.frame(cbind(  round(pl_1_wins,4),
 ↪  round(pl_2_wins,4),round(ties,4) ))
rownames(mcts_freq)=alphas_
colnames(mcts_freq)=cols
kable( mcts_freq,
  caption="Relative Frequencies Comparison")
```

Table 3: Relative Frequencies Comparison

	Pl 1 Win %	Pl 2 Win %	Ties %
0.95	0.2003	0.2202	0.0795
1	0.1973	0.2211	0.0816

At $\alpha = 0.95$, the estimated frequencies are relatively close to those of the full game tree, which is expected as alpha is fairely close to recovering the true frequencies from the whole game tree ($\alpha = 1$).

```
#| fig-cap: "Relative Game Frequencies"
#| fig-subcap:
#|   - "Distribution of Player 1 Win Freq"
#|   - "Distribution of Player 2 Win Freq"
#|   - "Distribution of Tie Game Freq"
#| layout-ncol: 3

#0.9
m0=as.matrix(c(+1, +1,-1, 0, 0, 0, 0, 0,-1))
```

```
N_sim=100
G_alpha_9=matrix(NA,nrow=N_sim,ncol=3)
for(i in 1:N_sim){
  g_i=mcts(m0,-1,alpha=0.9)
  num_T=length(g_i)
  win_per_pl1= length(which(g_i==-1))/num_T # %player
 ↪  1 wins
  G_alpha_9[i,1]=win_per_pl1
  win_per_pl2= length(which(g_i==1))/num_T# %player 2
 ↪  wins
  G_alpha_9[i,2]=win_per_pl2
  tie_per= length(which(g_i==0))/num_T# %ties
  G_alpha_9[i,3]=tie_per
}

#0.7

G_alpha_7=matrix(NA,nrow=N_sim,ncol=3)
for(i in 1:N_sim){
  g_i=mcts(m0,-1,alpha=0.7)
  num_T=length(g_i)
  win_per_pl1= length(which(g_i==-1))/num_T # %player
 ↪  1 wins
  G_alpha_7[i,1]=win_per_pl1
  win_per_pl2= length(which(g_i==1))/num_T# %player 2
 ↪  wins
  G_alpha_7[i,2]=win_per_pl2
  tie_per= length(which(g_i==0))/num_T# %ties
  G_alpha_7[i,3]=tie_per
}


# True vals

G_true=matrix(NA,nrow=N_sim,ncol=3)
for(i in 1:N_sim){
```

```
  g_i=game_tree(m0,-1)
  num_T=length(g_i)
  win_per_pl1= length(which(g_i==-1))/num_T # %player
 ↪  1 wins
  G_true[i,1]=win_per_pl1
  win_per_pl2= length(which(g_i==1))/num_T# %player 2
 ↪  wins
  G_true[i,2]=win_per_pl2
  tie_per= length(which(g_i==0))/num_T# %ties
  G_true[i,3]=tie_per
  }


actuals=apply(G_true,2,mean)

col_names=c('Player_1_Win_Freq','Player_2_Win_Freq',
 ↪  'Tie_Freq','alpha')
G1=data.frame(cbind(G_alpha_9,as.matrix(rep(0.9,dim(G_alpha_9)[1]))))
 ↪

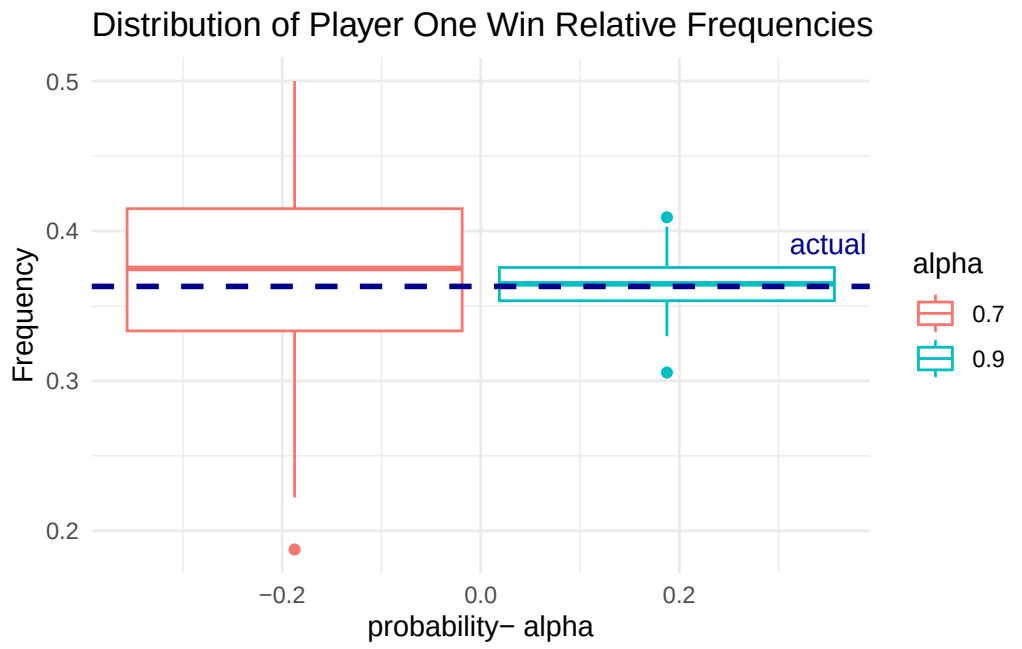
G2=data.frame(cbind(G_alpha_7,as.matrix(rep(0.7,dim(G_alpha_7)[1]))))
 ↪

G_all=rbind(G1,G2)
colnames(G_all)=col_names
G_all$alpha=as.character(G_all$alpha)



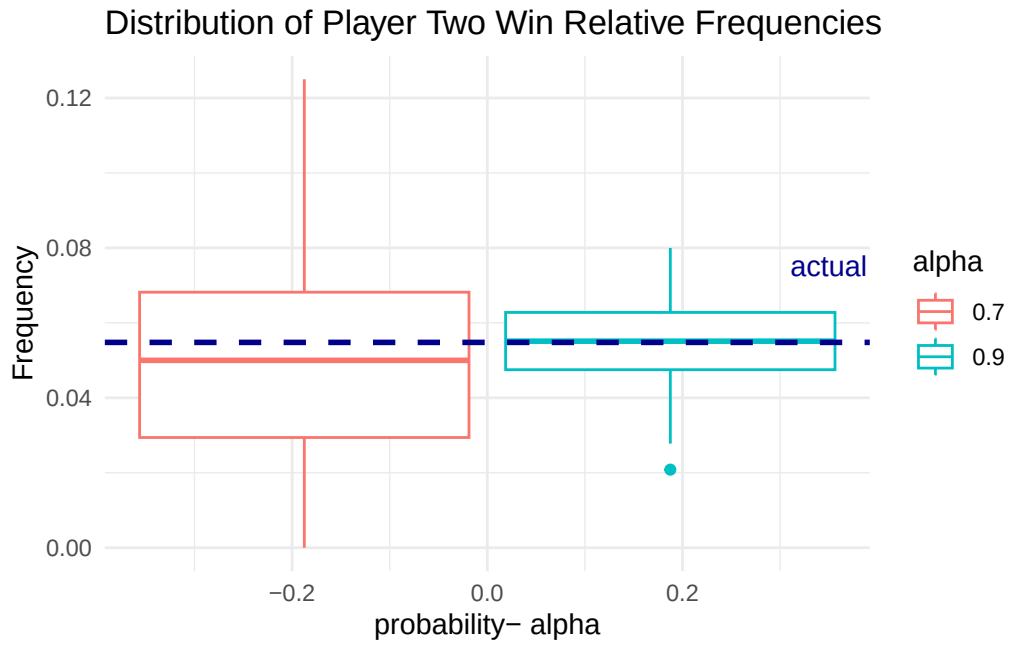

# Plot All simulations for different alphas
# Player 1 wins
G_all %>% ggplot(aes(y=Player_1_Win_Freq,col=alpha))+
  geom_boxplot()+
  labs(
    title='Distribution of Player One Win Relative
     ↪  Frequencies',
```

```
    x='probability- alpha',
    y='Frequency'
  )+
  geom_hline(yintercept = actuals[1], color =
   ↪  "darkblue", linetype = "dashed", linewidth = 1)
   ↪  +
 annotate("text", x = 0.35, y = actuals[1]+0.015,
  ↪  label = "actual",
          vjust = -0.5, color = "darkblue")+
   theme_minimal()
```



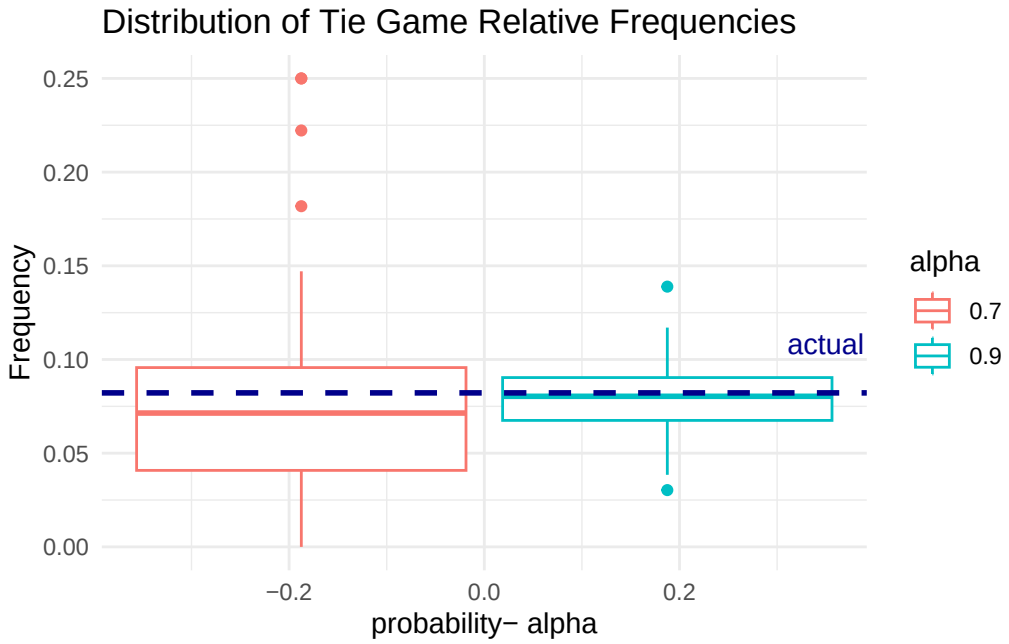

```
 # Player 2 wins
G_all %>% ggplot(aes(y=Player_2_Win_Freq,col=alpha))+
  geom_boxplot()+
  labs(
    title='Distribution of Player Two Win Relative
     ↪  Frequencies',
    x='probability- alpha',
```

```
    y='Frequency'
  )+
  geom_hline(yintercept = actuals[2], color =
   ↪  "darkblue", linetype = "dashed", linewidth = 1)
   ↪  +
 annotate("text", x = 0.35, y = actuals[2]+0.015,
  ↪  label = "actual",
          vjust = -0.5, color = "darkblue")+
   theme_minimal()
```




```
# Ties
G_all %>% ggplot(aes(y=Tie_Freq,col=alpha))+
  geom_boxplot()+
  labs(
    title='Distribution of Tie Game Relative
     ↪  Frequencies',
    x='probability- alpha',
    y='Frequency'
```

```
  )+
  geom_hline(yintercept = actuals[3], color =
   ↪  "darkblue", linetype = "dashed", linewidth = 1)
   ↪  +
annotate("text", x = 0.35, y = actuals[3]+0.015,
 ↪  label = "actual",
         vjust = -0.5, color = "darkblue")+
  theme_minimal()
```



Key Observations:

- Across box plots for the three relative frequencies, the distributions about the median for are tighter for $\alpha = 0.9$, compared to $\alpha = 0.7$. There is less variation in the frequency estimates across simulations for alpha values closer to 1 (the full game tree).
- As expected, the median relative frequencies across all distributions are also closer to their corresponding true values at the higher α value.

- **Conclusion**: The higher the α parameter, the closer MCTS is at recovering the full game's true realtive frequencies.

Soft-Margin SVM

a)

```
# kernal function to use the polynomial kernal trick
KK = function(X1,X2,cf,gm,dg)
 {
     return((cf+gm*t(X1)%*%X2)^dg)
}

my_svm=function(y,X,poly,Cost=10000){ # poly- vector
 ↪  of polynomial arguments c(gm,cf,dg)

  gm = poly[1] # coeff
  cf = poly[2] #constant
  dg = poly[3] # order of polynomial
  N=dim(X)[1]
  DD=matrix(0,N,N)

  # Fill in D mat
  for(i in 1:N)
 {
     for(j in 1:N)

     {
         DD[i,j] =
 ↪  y[i]*y[j]*KK(X[i,],X[j,],cf=cf,gm=gm,dg=dg) #
 ↪  Second part of Lagragian  formulation yn*ym*zn*zm
     }
  }
   eps = 5e-6
   DD  = DD +eps*diag(N)
```

```
 # Constraints LHS
 cons_1=diag(N);dim(cons_1)#lower bound of inequlity
↪  alpha>0
 cons_2=-diag(N);dim(cons_2)#upper bound of
↪  inequality aplha< C
 const_3=matrix(y,ncol=1,nrow=N);dim(const_3) #
↪  equality constraint alpha%*%yn=0
 Amat = cbind(const_3,cons_1,cons_2);dim(Amat) # N
↪  rows, M constraints
 d    =  matrix(1,N,1);dim(d) # coeffiencents of all
↪  alpha is 1, d in formulation

 # RHS of constraints
 rhs1=rep(0,N) # lower bound
 rhs2=rep(-Cost,N)# upper bound
 rhs3=0 # equality cons
 b0=c(rhs3,rhs1,rhs2)
 bvec=b0 # RHS

 # Solve the opt problem
  res = solve.QP(DD, d, Amat, bvec, meq=1,
↪  factorized=FALSE)
  a   = res$solution

  # Get wieghts
   # Recover the model:

    pad.a= round(a,3)
    wh = which.max(a) #

    # Find the ws:
    ww = t(a*y)%*%X # work back the wieghts/ X be
↪  substituted by the kernel val
    intercept =
↪  1/y[wh]-sum(a*y*KK(X[wh,],t(X),cf=cf,gm=gm,
↪  dg=dg)) # get intercept back
```

```
      T1 = rep(0,N)
      for(i in 1:N)
      {
        T1[i] = sum(a*y*KK(X[i,], t(X), cf=cf, gm=gm,
 ↪  dg=dg)) # matrix to get predictions/plane
      }

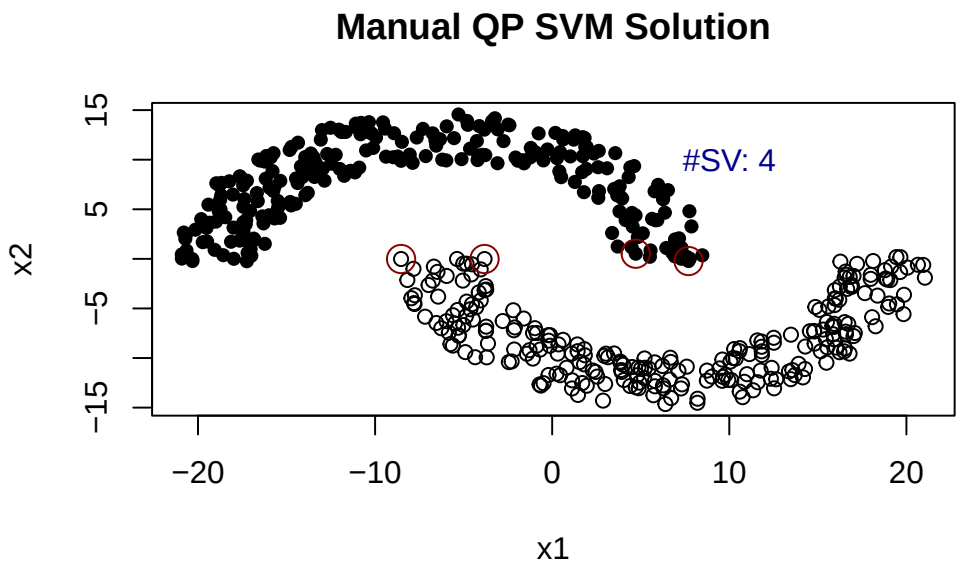
       yhat = sign(T1+intercept[1])

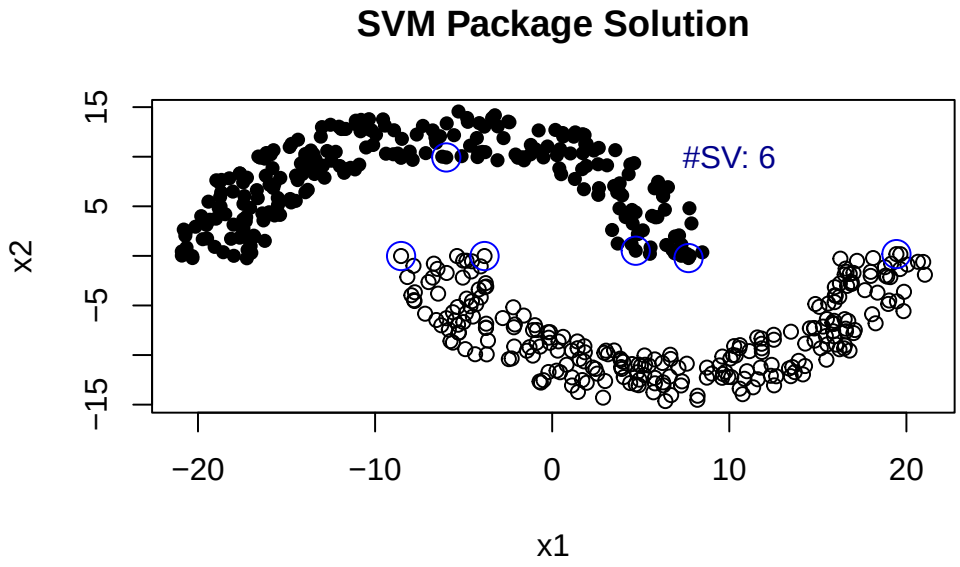
       # return list of a,weights,intercept,yhat

 ↪  ret=list(alphas=a,ww=ww,intercept=intercept,yhat=yhat)

      return(ret)
}
```

b)





Hyperparameters: **cost=100*, degree=3, gamma=1, coefficient=1.**

The number of support vectors found by the two methods equals 7. There is also an overlap with the data point support vectors found by the QP method and the svm package function i.e., all the support vectors from the former method are contained in the latter method and vice versa.
*The svm function ran out of iterations for higher cost values, which results in suboptimal solutions.

Appendix

Question 1 Code

```
#rm(list = ls())
#==========================================================================
#                  1. Set up a game universe
 ↪  (Encoding)
#==========================================================================
```

```
draw_objects = function(J)
{
     r_o        =  runif(J,0.3,1)
   pi_vals    =  runif(J,-1,1)*pi
   o1         =  r_o*cos(pi_vals)      # x-coordinates
 ↪  of objects
   o2         =  r_o*sin(pi_vals)
     return(cbind(o1,o2))
}
draw_starts = function(N = 1)
{
    r_x       =  runif(N,0,0.25)
    pi_x      =  runif(N,-1,1)*pi
    xt        =
 ↪  matrix(cbind(r_x*cos(pi_x),r_x*sin(pi_x)),nrow =
 ↪  N,byrow = TRUE)
}

set.seed(2024)

J       =  50
objects = draw_objects(J)
xt      = draw_starts()
delt    = 0.025              # How big are the steps
 ↪  you can take?
rad     = 0.05               # How close to an object
 ↪  before you crash?



plot_game = function(xt,objects,rad,cols = 1)
{
  plot(objects[,2]~objects[,1],type = 'n', ylim =
   ↪  c(-1,1), xlim = c(-1,1))
  symbols(objects[,1],objects[,2],circles =
   ↪  rep(rad,J),ylim = c(-1,1),xlim = c(-1,1),inches
   ↪  = FALSE,add = TRUE,bg = 'seagreen')
```

```
  points(xt[,2]~xt[,1], pch = 16, cex = 1,col = cols)
  pi_v = seq(-1,1,1/100)*pi
  y_edge = sin(pi_v)
  x_edge = cos(pi_v)
  lines(y_edge~x_edge)
  lines(c(0.25*y_edge)~c(0.25*x_edge),lty = 2)
}

plot_game(xt,objects,rad,'black')

set.seed(2024)
outer_rad=1
inner_rad=0.25

J       =   50 # num trees/objects to avoid
r_o       =  runif(J,0.3,1)
pi_vals   =  runif(J,-1,1)*pi
o1        =  r_o*cos(pi_vals)     # x-coordinates of
 ↪  objects
o2        =  r_o*sin(pi_vals)
objects =  cbind(o1,o2)


inner_rad=0.25
outer_rad=1
delt    =  0.025            # How big are the steps
 ↪  you can take?
rad     =  0.05 # How close to an object before you
 ↪  crash



draw_starts = function(N = 1)
{
    r_x      =  runif(N,0,0.25)
    pi_x     =  runif(N,-1,1)*pi
```

```
    xt       =
 ↪  matrix(cbind(r_x*cos(pi_x),r_x*sin(pi_x)),nrow =
 ↪  N,byrow = TRUE)
    return(xt)
}

# Initialize xt such that it is in the starting
 ↪  position inside the inner circle
x0=draw_starts(1)

set.seed(2024)

game_status = function(xt, objects, rad, center =
 ↪  matrix(c(0,0), nrow=1, ncol=2), outer_rad = 1)
{
  J        = nrow(objects)
  ones     = matrix(1, J, 1)
  dists    = sqrt(rowSums((objects - ones %*% xt)^2))
 ↪  # distance btwn drone and ALL obj
  min_dist = min(dists) # dist to closest tree

  outer_circle_point_dist = sqrt(sum((center -
 ↪  xt)^2)) # distance between circle center and
 ↪  drone

  if (min_dist < rad) {
    status = -1                        # hit a tree/obj
  } else if (outer_circle_point_dist > outer_rad) {
    status = 1                         # escaped
  } else {
    status = 0                         # keep going
  }

  list(status = status, closest =
   ↪  objects[which.min(dists), , drop = FALSE],
   ↪  min_dist = min_dist)
```

```
}

#game_status(x0, objects, rad, center =
 ↪  matrix(c(0,0), nrow=1, ncol=2), outer_rad = 1)


#| layout-ncol: 2
#| fig-subcap:
#|   - "GA Parameter Search Trace Plot"
#|   - "Trajectories over 100 initial positions"

set.seed(2024)

draw_starts = function(N = 1)
{
    r_x      =  runif(N,0,0.25)
    pi_x     =  runif(N,-1,1)*pi
    xt       =
 ↪  matrix(cbind(r_x*cos(pi_x),r_x*sin(pi_x)),nrow =
 ↪  N,byrow = TRUE)
    return(xt)
}

p     = 2
q     = 2
nodes =c(3,3)
npars =
 ↪  p*nodes[1]+nodes[1]*nodes[2]*+nodes[2]*q+nodes[1]+nodes[2]+q
 ↪  # total num param
theta_rand = runif(npars,-1,1)

#####################NN
 ↪  MOdel##############################

model = function(xt, theta, nodes = c(3,3))
{
```

```
  # Infer dimensions:
  N = dim(xt)[1]
  p = dim(xt)[2]    # number of input features/coords
  q = 2             # output steps
  dims = c(p, nodes, q)    # p -> 3 -> 3 -> 2

  # Populate weight matrices:
  index = 1:(dims[1]*dims[2])
  W1    = matrix(theta[index], dims[1], dims[2])
  index = max(index) + 1:(dims[2]*dims[3])
  W2    = matrix(theta[index], dims[2], dims[3])
  index = max(index) + 1:(dims[3]*dims[4])
  W3    = matrix(theta[index], dims[3], dims[4])

  # Populate bias vectors:
  index = max(index) + 1:dims[2]
  b1    = matrix(theta[index], dims[2], 1)
  index = max(index) + 1:dims[3]
  b2    = matrix(theta[index], dims[3], 1)
  index = max(index) + 1:dims[4]
  b3    = matrix(theta[index], dims[4], 1)

  ones = matrix(1, N, 1)
  a0   = t(xt)

  # Evaluate the updating equation in matrix form
  a1 = tanh(t(W1) %*% a0 + b1 %*% t(ones))    # hidden
 ↪  layer 1
  a2_hidd = tanh(t(W2) %*% a1 + b2 %*% t(ones))    #
 ↪  hidden layer 2
  a3 = tanh(t(W3) %*% a2_hidd + b3 %*% t(ones))    #
 ↪  output layer

  # Return a list of relevant objects:
  return(list(a2 = matrix(a3, ncol = 2, nrow = 1)))
}
```

```
#################### plot game function
plot_game =
 ↪  function(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2),inner
{
  plot(NULL, type ='n', col =
   ↪  'darkgreen',xlim=c(-2,2),ylim=c(-2,2))

  # plot inner and outer circles

  theta = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(theta)
  y_inner = center[2] + inner_rad * sin(theta)


  x_outer = center[1] + outer_rad * cos(theta)
  y_outer = center[2] + outer_rad * sin(theta)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

  points(xt[,2]~xt[,1],pch = 9,col = 'black')

   ↪  points(objects[,1],objects[,2],pch=19,col='darkgreen')
  abline(v = c(-1,1), lty = 3,col = 'black',lwd = 2)
  abline(h = c(-1,1), lty = 3,col =
   ↪  c('black','black'), lwd = 2)
}


#################################### game_play
```

```
# play a single game

play_a_game = function(xt,delt,objects,rad,theta,plt
 ↪  = F,trace =
 ↪  TRUE,nodes,inner_rad=0.25,outer_rad=1)
{
  k           =   1  # Count how many iterations
  xt          =   xt # Set the initial coordinate(s)
 ↪  for the drone.
  trajectory = matrix(0,2,100) # store the the
 ↪  postions that the drone takes through out the
 ↪  game/how it moves
  trajectory[,1] = t(x0)

  # Check the game status:
  res_status  = game_status(xt,objects,rad)

  if(plt)
  {

       ↪  plot_game(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2
  }
  while((res_status$status==0)&(k<100)) # single
   ↪  game=100 steps
  {
    k = k+1

    # Now, let the model update the position of the
     ↪  pieces:
    ct = model(xt,theta,nodes)$a2 #res_status$closest
    xt =  xt+ct*delt

    #print(paste0('k: ',k,'xt_new: ',xt))
```

```
    # Checkk the game status after the positions are
     ↪ updates:

    res_status  = game_status(xt,objects,rad)

    trajectory[,k] = t(xt)

  }
  trajectory = trajectory[,1:k,drop = FALSE]
    if(plt)
  {
    lines(trajectory[2,]~trajectory[1,],col =
     ↪ 'darkgrey')
  }

  dmin_final=sqrt(sum((res_status$closest - xt)^2))

  return(list(k = k, status = res_status$status,xt=
   ↪ xt,trajectory = trajectory,dmin=dmin_final))
}

#play_a_game(draw_starts(1),delt,objects,rad,theta_rand,nodes=c(3,3),
 ↪ = F)



############ Obj func
play_T_games = function(theta) # This is our obj
 ↪ function!
{
  inner_rad_in=0.25;outer_rad_in=1;T_games=100;J=50

     # num trees/objects to avoid
  score=0
    for(i in 1:T_games)
    {
```

```
  objects =  objects # keeo constant for now
 #starting position
    xt_try=draw_starts(1)
    #apply across x_try
   res_play
 ↪  =play_a_game(xt_try,delt,objects,rad,theta,plt =
 ↪  FALSE,trace =
 ↪  FALSE,nodes=c(3,3),inner_rad=inner_rad_in,outer_rad=outer_rad_in)
   score  = score+(res_play==+1) # number of times
 ↪  when we got to the other side
    }
  return(score/T_games) # objective to maximise
}

###################GA function to get
 ↪  weights#################
# Use GA function

library('GA')

# Read saved RDS From GA code below

# GA <- ga(type   = 'real-valued',
#          fitness = play_T_games, # used here to get
 ↪  the optimal theta
#          lower   = rep(-10,npars), # lower bound
 ↪  vector
#          upper   = rep(+10,npars), # upper bound
 ↪  vect
#          popSize = 100,
#          maxiter = 200) # nr generations
#


#saveRDS(GA,file='ga_results2.rds')
```

```
GA=readRDS('ga_results2.rds')
plot(GA)
theta_opt=GA@solution[1,]

# Draw race over 100 random starts
tries=draw_starts(100)
center=c(0,0)

plot(NULL, xlim=c(-2,2),ylim=c(-2,2),xlab='position
 ↪  coord 1',ylab='position coord 2',main='Trace Plot
 ↪  for 100 Trajectories: Fixed Tree Configuration')

  thetas = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(thetas)
  y_inner = center[2] + inner_rad * sin(thetas)


  x_outer = center[1] + outer_rad * cos(thetas)
  y_outer = center[2] + outer_rad * sin(thetas)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

  # points(xt[,2]~xt[,1],pch = 9,col = 'black')

   ↪  points(objects[,1],objects[,2],pch=19,col='darkgreen')


# play the game 100 times with the optimal theta
scores=c() # store results of each game
for(i in 1:dim(tries)[1]){
  xt_i=matrix(tries[i,],nrow=1,ncol=2)

 ↪  res=play_a_game(xt_i,delt,objects,rad,theta=theta_opt,plt
 ↪  = F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
```

```
 scores=c(scores,res$status)
  traj= res$trajectory
  colr=ifelse(res$status==1,'lightgreen','brown') #
 ↪  green if succ
  lines(traj[1,]~traj[2,],lwd=1.5,col=colr)

  }

# %Successful exits
sr=(length(which(scores==1))/length(scores))*100 #98%

text(x = 1.5, y = 1.5, labels = paste("Succ
 ↪  Rate:~",round(sr,3),'%'), col = "darkblue", cex =
 ↪  1, font = 1)




# New Trees

outer_rad=1
inner_rad=0.25

J       =   50

set.seed(124)

draw_objects = function(J)
{
     r_o       =  runif(J,0.3,1)
   pi_vals    =  runif(J,-1,1)*pi
   o1         =  r_o*cos(pi_vals)      # x-coordinates
 ↪  of objects
   o2         =  r_o*sin(pi_vals)
     return(cbind(o1,o2))
```

```
}


new_obj=draw_objects(J)

win = c()


tries=draw_starts(100)
center=c(0,0)

plot(NULL, xlim=c(-2,2),ylim=c(-2,2),xlab='position
 ↪  coord 1',ylab='position coord 2',main='Trace Plot
 ↪  for 100 Trajectories: New Tree Config')

  thetas = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(thetas)
  y_inner = center[2] + inner_rad * sin(thetas)


  x_outer = center[1] + outer_rad * cos(thetas)
  y_outer = center[2] + outer_rad * sin(thetas)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

  # points(xt[,2]~xt[,1],pch = 9,col = 'black')

   ↪  points(new_obj[,1],new_obj[,2],pch=19,col='darkgreen')


# play the game 100 times with the optimal theta
scores=c() # store results of each game
```

```
for(i in 1:dim(tries)[1]){
  xt_i=matrix(tries[i,],nrow=1,ncol=2)

 ↪  res=play_a_game(xt_i,delt,new_obj,rad,theta=theta_opt,plt
 ↪  = F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
 scores=c(scores,res$status)
  traj= res$trajectory
  colr=ifelse(res$status==1,'lightgreen','brown') #
 ↪  green if succ
  lines(traj[1,]~traj[2,],lwd=1.5,col=colr)

  }

# %Successful exits
sr=round(
 ↪  (length(which(scores==1))/length(scores)),3)

text(x = 1.5, y = 1.5, labels = paste("Succ
 ↪  Rate:~",sr), col = "darkblue", cex = 1, font = 1)
```

```
model_2 = function(xt_input, theta, nodes =
 ↪  c(3,3),add_arg) #add_arg= 1/(dmin-r)
{
  xt=cbind(xt_input,add_arg)
  # Infer dimensions:
  N = dim(xt)[1]
  p = dim(xt)[2]   # number of input features/coords
  q = 2            # output steps
```

```
  dims = c(p, nodes, q)    # p -> 3 -> 3 -> 2

  # Populate weight matrices:
  index = 1:(dims[1]*dims[2])
  W1    = matrix(theta[index], dims[1], dims[2])
  index = max(index) + 1:(dims[2]*dims[3])
  W2    = matrix(theta[index], dims[2], dims[3])
  index = max(index) + 1:(dims[3]*dims[4])
  W3    = matrix(theta[index], dims[3], dims[4])

  # Populate bias vectors:
  index = max(index) + 1:dims[2]
  b1    = matrix(theta[index], dims[2], 1)
  index = max(index) + 1:dims[3]
  b2    = matrix(theta[index], dims[3], 1)
  index = max(index) + 1:dims[4]
  b3    = matrix(theta[index], dims[4], 1)

  ones = matrix(1, N, 1)
  a0   = t(xt)

  # Evaluate the updating equation in matrix form
  a1 = tanh(t(W1) %*% a0 + b1 %*% t(ones))    # hidden
 ↪  layer 1
  a2_hidd = tanh(t(W2) %*% a1 + b2 %*% t(ones))    #
 ↪  hidden layer 2
  a3 = tanh(t(W3) %*% a2_hidd + b3 %*% t(ones))    #
 ↪  output layer

  # Return a list of relevant objects:
  return(list(a2 = matrix(a3, ncol = 2, nrow = 1)))
}


# Modify play_a_game function
# now includes d_min
```

```
play_a_game_mod =
 ↪  function(xt,delt,objects,rad,theta,plt = F,trace
 ↪  = TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
{
  k           =   1  # Count how many iterations
  xt          =   xt # Set the initial coordinate(s)
 ↪  for the drone.
  trajectory = matrix(0,2,100) # store the the
 ↪  postions that the drone takes through out the
 ↪  game/how it moves
  trajectory[,1] = t(xt)

  # Check the game status:
  res_status  = game_status(xt,objects,rad)
  d_min=sqrt(sum((res_status$closest - xt)^2))

  if(plt)
  {

       ↪  plot_game(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2
  }
  while((res_status$status==0)&(k<100)) # single
   ↪  game=100 steps
  {
    k = k+1

    # Now, let the model update the position of the
     ↪  pieces:
    add_arg=(1/(d_min-rad))
    ct = model_2(xt,theta,nodes,add_arg)$a2
 ↪  #res_status$closest
    xt =  xt+ct*delt

    #print(paste0('k: ',k,'xt_new: ',xt))
```

```
    # Checkk the game status after the positions are
     ↪  updates:

    res_status  = game_status(xt,objects,rad)

    trajectory[,k] = t(xt)

  }
  trajectory = trajectory[,1:k,drop = FALSE]
    if(plt)
  {
    lines(trajectory[2,]~trajectory[1,],col =
     ↪  'darkred',lwd=2)
    #point(trajectory[2,]~trajectory[1,],col =
     ↪  'darkred',lwd=2)
    points(xt[,2]~xt[,1],pch = 9,col = 'black')
  }

  dmin_final=sqrt(sum((res_status$closest - xt)^2))

  return(list(k = k, status = res_status$status,xt=
   ↪  xt,trajectory =
   ↪  trajectory,dmin_final=dmin_final))
}

# Test for a single game
#play_a_game_mod(x0,delt,objects,rad,theta_opt,nodes=c(3,3),plt
 ↪  = T)


 # modified to make a new draw and tree config
play_a_game_mod2 = function(delt,rad,theta,plt =
 ↪  F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
```

```
{
  J=50
  # random start
  xt=draw_starts(1) # new random start
  objects =  objects=draw_objects(J)
  k           =   1  # Count how many iterations
  xt          =   xt # Set the initial coordinate(s)
 ↪  for the drone.
  trajectory = matrix(0,2,100) # store the the
 ↪  postions that the drone takes through out the
 ↪  game/how it moves
  trajectory[,1] = t(xt)

  # Check the game status:
  res_status  = game_status(xt,objects,rad)
  d_min=sqrt(sum((res_status$closest - xt)^2))


  if(plt)
  {

       ↪  plot_game(xt,objects,rad,center=matrix(c(0,0),nrow=1,ncol=2
  }
  while((res_status$status==0)&(k<100)) # single
   ↪  game=100 steps
  {
    k = k+1

    # Now, let the model update the position of the
     ↪  pieces:
    add_arg=(1/(d_min-rad))
    ct = model_2(xt,theta,nodes,add_arg)$a2
↪   #res_status$closest
    xt =  xt+ct*delt
```

```
    #print(paste0('k: ',k,'xt_new: ',xt))

    # Checkk the game status after the positions are
     ↪ updates:

    res_status  = game_status(xt,objects,rad)

    trajectory[,k] = t(xt)

  }
  trajectory = trajectory[,1:k,drop = FALSE]
    if(plt)
  {
    lines(trajectory[2,]~trajectory[1,],col =
     ↪ 'darkred',lwd=2)
    #point(trajectory[2,]~trajectory[1,],col =
     ↪ 'darkred',lwd=2)
    points(xt[,2]~xt[,1],pch = 9,col = 'black')
  }

  dmin_final=sqrt(sum((res_status$closest - xt)^2))

  return(list(k = k, status = res_status$status,xt=
   ↪ xt,trajectory =
   ↪ trajectory,dmin_final=dmin_final))
}


# Results
set.seed(46)
plot(NULL, xlim=c(-2,2),ylim=c(-2,2),xlab='position
 ↪ coord 1',ylab='position coord 2',main='100
 ↪ Trajectories: Varying Initial Starts and Tree
 ↪ Configs')
```

```
  thetas = seq(0, 2 * pi, length.out = 200)

  x_inner = center[1] + inner_rad * cos(thetas)
  y_inner = center[2] + inner_rad * sin(thetas)


  x_outer = center[1] + outer_rad * cos(thetas)
  y_outer = center[2] + outer_rad * sin(thetas)


  lines(x_outer, y_outer, col = "purple", lwd = 2)
  lines(x_inner, y_inner, col = "blue", lwd = 2)
  abline(h = 0, v = 0, col = "gray", lty = 2)

  # points(xt[,2]~xt[,1],pch = 9,col = 'black')

# play the game 100 times with the optimal theta
scores=c() # store results of each game
for(i in 1:dim(tries)[1]){
  res=play_a_game_mod2(delt,rad,theta=theta_opt,plt =
 ↪  F,trace =
 ↪  TRUE,nodes=c(3,3),inner_rad=0.25,outer_rad=1)
 scores=c(scores,res$status)
  traj= res$trajectory
  colr=ifelse(res$status==1,'lightgreen','brown') #
 ↪  green if succ
  lines(traj[1,]~traj[2,],lwd=1.5,col=colr)


  }

# %Successful exits
sr=round(
 ↪  (length(which(scores==1))/length(scores)),3)

text(x = 0.9, y = 1.5, labels = paste("Succ
 ↪  Rate:~",sr), col = "darkblue", cex = 1, font = 1)
```

Question 2 Code

```
rm(list = ls())
game_tree= function(m,k)
{
    g = c()
    game_state = rho(m,S)
    if(game_state$terminal)
    {
         g = c(g,game_state$winner)
         return(g)
    }else{
        index = which(m == 0) # open slot on the game
 ↪  board/ vector of open slots
        for(i in 1:length(index))
        {
            x           = m # how the board looks
 ↪  before the next move
            x[index[i]] = k # next player's turn
            g           = c(g,game_tree(x,-1*k)) #
 ↪  recursive call, append outcome to g
        }
        return(g)
    }
}
# Define game status


#=====================================================================
# Populate the state matrix
#=====================================================================
S = matrix(0,9,8)
# Row Sums:
S[1:3,1] = 1
```

```
S[4:6,2] = 1
S[7:9,3] = 1
# Col Sums:
S[c(1,4,7),4] = 1
S[c(2,5,8),5] = 1
S[c(3,6,9),6] = 1
# Diag Sums
S[c(1,5,9),7] = 1
S[c(3,5,7),8] = 1



rho = function(m,S)
{
  m = matrix(m,ncol = 1)
  player1  = any(t(m)%*%S == -3) # player X has won
 ↪  along any of the rows/col
  player2  = any(t(m)%*%S == +3) # player O has won
 ↪  along any of the rows/col
  m_1      = (m == -1) # player X entries on the
 ↪  board index /vector of booleans/ 1s an 0s
  m_2      = (m == +1) # player O entries on the
 ↪  board index /vector of booleans/ 1s an 0s
  tie      = sum((t(m_1)%*%S>0)*(t(m_2)%*%S>0)) == 8
 ↪  # check for draw
  winner   = c(-1,0,+1,NA)[c(player1,tie,player2,NA)]
 ↪  # indicate what game outcome is
  terminal = player1|tie|player2 # check if game HAS
 ↪  ended tie to the above conditions, returns FALSE
 ↪  if not
  ret      = list(terminal = terminal,winner =
 ↪  winner)
  return(ret)
}
```

```
# m0 = as.matrix(c(0,0,0,0,0,0,0,0,0)) # current game
 ↪  board/flattened
# c(rho(m0,S)$winner ,1)

# Mofified game tree
game_tree_mod= function(m,k,n_moves,strict=F,n_sim=1)
 ↪  #n_moves=target moves
{
    g = c()

    for(sim in 1:n_sim){
    x=m
    game_state = rho(x,S)

    if(strict){
    condition=(n_moves<=0)}else{ # exactly condition
      condition=((game_state$terminal)|(n_moves<=0))
 ↪  # up to condition
    }

    if(condition){
         g = c(g,game_state$winner)
         return(g)
    }else{
        index_positions = which(m == 0) # open slot
 ↪  on the game board/ vector of open

        for(i in 1:length(index_positions))
        {
          x=m
            picked_index=index_positions[i]

            x[picked_index] = k # player's turn
```

```
            g  =c(g,
 ↪  game_tree_mod(x,-1*k,n_moves-1,n_sim=1) )

        }
        }
        return(g)
    }
    }


# Number wins in 5 moves
m0=as.matrix(rep(0,9))# open board


game_Ts_5=game_tree_mod(m0,1,n_moves=5,strict=T) # Ts
 ↪  for exactly 5 moves
num_Ts_5=length(game_Ts_5)
num_wins_5=length(which(
 ↪  (game_Ts_5==1)|(game_Ts_5==-1)   )) # across
 ↪  players
num_wins_5;length(game_Ts_5) # player 1 wins all the
 ↪  time if moves are constrained to 5 in total, 12
 ↪  wins across trajectories

m0=as.matrix(rep(0,9))# open board

game_Ts_8=game_tree_mod(m0,1,n_moves=8,strict=T) #
 ↪  Ts for exactly 8 moves
num_Ts_8=length(game_Ts_8)
num_wins_8=length(which(
 ↪  (game_Ts_8==1)|(game_Ts_8==-1)   )) # just wins
 ↪  for either player
num_wins_8;length(game_Ts_8)
 ↪  ;num_wins_8/length(game_Ts_8)
```

```
# players 89% of the time when moves are constrained
 ↪  to 8 excatly
# Player one vs player 2:
# player 1 wins 10.5K times, player 2 wines 9K times

m0=as.matrix(rep(0,9))# open board
game_Ts_atmost_8=game_tree_mod(m0,1,n_moves=8,strict=F)
 ↪  # up to 8 moves
num_Ts_atmost_8=length(game_Ts_atmost_8)
num_wins_atmost_8=length(which(
 ↪  (game_Ts_atmost_8==1)|(game_Ts_atmost_8==-1)   ))
num_wins_atmost_8
pl_1_num_wins_atmost_8=length(which(game_Ts_atmost_8==1))
pl_2_num_wins_atmost_8=length(which(game_Ts_atmost_8==-1))


game_config=c('5 exact moves','8 exact moves')
metrics=c('#Game Trajectories','Total Wins')
total_game_trajectories=c(num_Ts_5,num_Ts_8)
total_wins=c(num_wins_5,num_wins_8)

summ_df=data.frame(rbind(total_game_trajectories,total_wins))
colnames(summ_df)=game_config
rownames(summ_df)=metrics
kable(summ_df,caption = "Total Wins: 5 vs 8 move game
 ↪  (exact)" )




# Up to 8 moves
players=c('player 1','player 2')
wins=c(pl_1_num_wins_atmost_8,pl_2_num_wins_atmost_8)
win_percentage=c(pl_1_num_wins_atmost_8,pl_2_num_wins_atmost_8)/num_T

summ_df2=data.frame(wins)
```

```
rownames(summ_df2)=players
summ_df2$percentage=round(win_percentage,4)
kable(summ_df2,caption = "Player Wins For (up to) 8
 ↪  move game")


mcts= function(m,k,alpha,n_sim=1)
{

  g = c()

    for(sim in 1:n_sim){
    x=m
    game_state = rho(x,S)

    if(game_state$terminal)
    {
         g = c(g,game_state$winner)
         return(g)
    }else{
        index_positions = which(m == 0) # open slot
 ↪  on the game board/ vector of open

        for(i in 1:length(index_positions))
        {
          if (runif(1,0,1) <= alpha) {
          x=m
            picked_index=index_positions[i]

            x[picked_index] = k # player's turn
            g  =c(g, mcts(x,k*-1,alpha) )


          }
        }
        }
```

```
        return(g)
    }
    }


# Table Summ

cols=c('Pl 1 Win %','Pl 2 Win %','Ties %')
alphas_=c(0.95,1)

m0=as.matrix(c(+1, +1, 0, 0, 0, 0, 0,0,-1))

pl_1_wins=c()
pl_2_wins=c()
ties=c()
for(alph in alphas_ ){
  if(alph!=1){
  g_sample=mcts(m0,-1,alph)
  }else{
    g_sample=game_tree(m0,-1)
  }

  num_traj=length(g_sample)
  num_wins_player1=round( (length(
 ↪  which(g_sample==-1))/num_traj),4)
  num_wins_player2=length(
 ↪  which(g_sample==1))/num_traj
  num_ties=length(which(g_sample==0))/num_traj
  pl_1_wins=c(pl_1_wins,num_wins_player1)
  pl_2_wins=c(pl_2_wins,num_wins_player2)
  ties=c(ties,num_ties)
}

mcts_freq=data.frame(cbind(  round(pl_1_wins,4),
 ↪  round(pl_2_wins,4),round(ties,4) ))
rownames(mcts_freq)=alphas_
```

```
colnames(mcts_freq)=cols
kable( mcts_freq,
  caption="Relative Frequencies Comparison")



#| fig-cap: "Relative Game Frequencies"
#| fig-subcap:
#|   - "Distribution of Player 1 Win Freq"
#|   - "Distribution of Player 2 Win Freq"
#|   - "Distribution of Tie Game Freq"
#| layout-ncol: 3

#0.9
m0=as.matrix(c(+1, +1,-1, 0, 0, 0, 0, 0,-1))
N_sim=100
G_alpha_9=matrix(NA,nrow=N_sim,ncol=3)
for(i in 1:N_sim){
  g_i=mcts(m0,-1,alpha=0.9)
  num_T=length(g_i)
  win_per_pl1= length(which(g_i==-1))/num_T # %player
 ↪  1 wins
  G_alpha_9[i,1]=win_per_pl1
  win_per_pl2= length(which(g_i==1))/num_T# %player 2
 ↪  wins
  G_alpha_9[i,2]=win_per_pl2
  tie_per= length(which(g_i==0))/num_T# %ties
  G_alpha_9[i,3]=tie_per
}

#0.7

G_alpha_7=matrix(NA,nrow=N_sim,ncol=3)
for(i in 1:N_sim){
  g_i=mcts(m0,-1,alpha=0.7)
  num_T=length(g_i)
```

```
  win_per_pl1= length(which(g_i==-1))/num_T # %player
 ↪  1 wins
  G_alpha_7[i,1]=win_per_pl1
  win_per_pl2= length(which(g_i==1))/num_T# %player 2
 ↪  wins
  G_alpha_7[i,2]=win_per_pl2
  tie_per= length(which(g_i==0))/num_T# %ties
  G_alpha_7[i,3]=tie_per
}


# True vals

G_true=matrix(NA,nrow=N_sim,ncol=3)
for(i in 1:N_sim){
  g_i=game_tree(m0,-1)
  num_T=length(g_i)
  win_per_pl1= length(which(g_i==-1))/num_T # %player
 ↪  1 wins
  G_true[i,1]=win_per_pl1
  win_per_pl2= length(which(g_i==1))/num_T# %player 2
 ↪  wins
  G_true[i,2]=win_per_pl2
  tie_per= length(which(g_i==0))/num_T# %ties
  G_true[i,3]=tie_per
  }


actuals=apply(G_true,2,mean)

col_names=c('Player_1_Win_Freq','Player_2_Win_Freq',
 ↪  'Tie_Freq','alpha')
G1=data.frame(cbind(G_alpha_9,as.matrix(rep(0.9,dim(G_alpha_9)[1]))))
 ↪

G2=data.frame(cbind(G_alpha_7,as.matrix(rep(0.7,dim(G_alpha_7)[1]))))
 ↪
```

```
G_all=rbind(G1,G2)
colnames(G_all)=col_names
G_all$alpha=as.character(G_all$alpha)



# Plot All simulations for different alphas
# Player 1 wins
G_all %>% ggplot(aes(y=Player_1_Win_Freq,col=alpha))+
  geom_boxplot()+
  labs(
    title='Distribution of Player One Win Relative
     ↪  Frequencies',
    x='probability- alpha',
    y='Frequency'
  )+
  geom_hline(yintercept = actuals[1], color =
   ↪  "darkblue", linetype = "dashed", linewidth = 1)
   ↪  +
 annotate("text", x = 0.35, y = actuals[1]+0.015,
  ↪  label = "actual",
          vjust = -0.5, color = "darkblue")+
   theme_minimal()



 # Player 2 wins
G_all %>% ggplot(aes(y=Player_2_Win_Freq,col=alpha))+
  geom_boxplot()+
  labs(
    title='Distribution of Player Two Win Relative
     ↪  Frequencies',
    x='probability- alpha',
    y='Frequency'
```

```
  )+
  geom_hline(yintercept = actuals[2], color =
   ↪  "darkblue", linetype = "dashed", linewidth = 1)
   ↪  +
 annotate("text", x = 0.35, y = actuals[2]+0.015,
  ↪  label = "actual",
          vjust = -0.5, color = "darkblue")+
   theme_minimal()

# Ties
G_all %>% ggplot(aes(y=Tie_Freq,col=alpha))+
  geom_boxplot()+
  labs(
    title='Distribution of Tie Game Relative
     ↪  Frequencies',
    x='probability- alpha',
    y='Frequency'
  )+
  geom_hline(yintercept = actuals[3], color =
   ↪  "darkblue", linetype = "dashed", linewidth = 1)
   ↪  +
 annotate("text", x = 0.35, y = actuals[3]+0.015,
  ↪  label = "actual",
          vjust = -0.5, color = "darkblue")+
   theme_minimal()
```

Question 3 Code

```
# kernal function to use the polynomial kernal trick
KK = function(X1,X2,cf,gm,dg)
 {
     return((cf+gm*t(X1)%*%X2)^dg)
}

my_svm=function(y,X,poly,Cost=10000){ # poly- vector
 ↪  of polynomial arguments c(gm,cf,dg)
```

```
 gm = poly[1] # coeff
 cf = poly[2] #constant
 dg = poly[3] # order of polynomial
 N=dim(X)[1]
 DD=matrix(0,N,N)

 # Fill in D mat
 for(i in 1:N)
{
    for(j in 1:N)

    {
        DD[i,j] =
↪  y[i]*y[j]*KK(X[i,],X[j,],cf=cf,gm=gm,dg=dg) #
↪  Second part of Lagragian  formulation yn*ym*zn*zm
    }
 }
  eps = 5e-6
  DD  = DD +eps*diag(N)

 # Constraints LHS
 cons_1=diag(N);dim(cons_1)#lower bound of inequlity
↪  alpha>0
 cons_2=-diag(N);dim(cons_2)#upper bound of
↪  inequality aplha< C
 const_3=matrix(y,ncol=1,nrow=N);dim(const_3) #
↪  equality constraint alpha%*%yn=0
 Amat = cbind(const_3,cons_1,cons_2);dim(Amat) # N
↪  rows, M constraints
 d    =  matrix(1,N,1);dim(d) # coeffiencents of all
↪  alpha is 1, d in formulation

 # RHS of constraints
 rhs1=rep(0,N) # lower bound
```

```
 rhs2=rep(-Cost,N)# upper bound
 rhs3=0 # equality cons
 b0=c(rhs3,rhs1,rhs2)
 bvec=b0 # RHS

 # Solve the opt problem
  res = solve.QP(DD, d, Amat, bvec, meq=1,
↪  factorized=FALSE)
  a   = res$solution

  # Get wieghts
   # Recover the model:

    pad.a= round(a,3)
    wh = which.max(a) #

    # Find the ws:
    ww = t(a*y)%*%X # work back the wieghts/ X be
↪  substituted by the kernel val
    intercept =
↪  1/y[wh]-sum(a*y*KK(X[wh,],t(X),cf=cf,gm=gm,
↪  dg=dg)) # get intercept back

     T1 = rep(0,N)
     for(i in 1:N)
     {
       T1[i] = sum(a*y*KK(X[i,], t(X), cf=cf, gm=gm,
↪  dg=dg)) # matrix to get predictions/plane
     }

      yhat = sign(T1+intercept[1])

      # return list of a,weights,intercept,yhat

↪  ret=list(alphas=a,ww=ww,intercept=intercept,yhat=yhat)
```

```
      return(ret)
}



plm_data=read.delim("PLA_Dynamics", header =
 ↪  TRUE,sep='')

y=plm_data$Y
x1=plm_data$X1
x2=plm_data$X2
X=cbind(x1,x2)
poly=c(0.8,1,3)

# Test my_svm
res=my_svm(y,X,poly,Cost=100)

#SV alpha coeff!=0
# Nr of SV
my_sv_coeff=which(res$alpha>1e-6) #alpha greater than
 ↪  tolerance
my_mod_sv=dim(X[my_sv_coeff,])[1]



gm = poly[1] # coeff
cf = poly[2] #constant
dg = poly[3] # order of polynomial

model  = svm(X,y, cost = 100, kernel = 'polynomial',
 ↪  coef0 = cf, degree = dg, gamma = gm,scale
 ↪  =F,type="C-classification")


svm_Sv=model$SV
svm_nr_SV=dim(svm_Sv)[1]
```

```
#| fig-cap: "QP SVM vs SVM Function"
#| fig-subcap:
#|   - "QP Support Vectors"
#|   - "SVM Support Vectors"
#| layout-ncol: 2


a   = res$alpha[my_sv_coeff]
plot(x2~x1,pch = c(1,16)[(y+1)/2+1],main='Manual QP
 ↪  SVM Solution')
points(x2[my_sv_coeff]~x1[my_sv_coeff], pch = 1,col =
 ↪  'darkred',cex=2)
text(x = 10, y = 10, labels =
 ↪  paste("#SV:",my_mod_sv), col = "darkblue", cex =
 ↪  1, font = )

# SVM SVs
plot(x2~x1,pch = c(1,16)[(y+1)/2+1], main='SVM
 ↪  Package Solution')
points(svm_Sv[,'x2']~svm_Sv[,'x1'], pch = 1,col =
 ↪  'blue',cex=2)
text(x = 10, y = 10, labels =
 ↪  paste("#SV:",svm_nr_SV), col = "darkblue", cex =
 ↪  1, font = )
```